

PEC 2: Juegos

Presentación

Segunda PEC del curso de Inteligencia Artificial I

Competencias

En esta PEC se trabajaran las siguientes competencias:

Competencias de grado:

- Capacidad de analizar un problema con el nivel de abstracción adecuado a cada situación y aplicar las habilidades y conocimientos adquiridos para abordarlo y solucionarlo.

Competencias específicas:

- Saber representar las particularidades de un problema según un modelo de representación del conocimiento.
- Saber resolver problemas intratables a partir de razonamientos aproximados y heurísticos (algoritmos voraces, algoritmos genéticos, lógica difusa, redes bayesianas, redes neuronales, min-max).

Objetivos

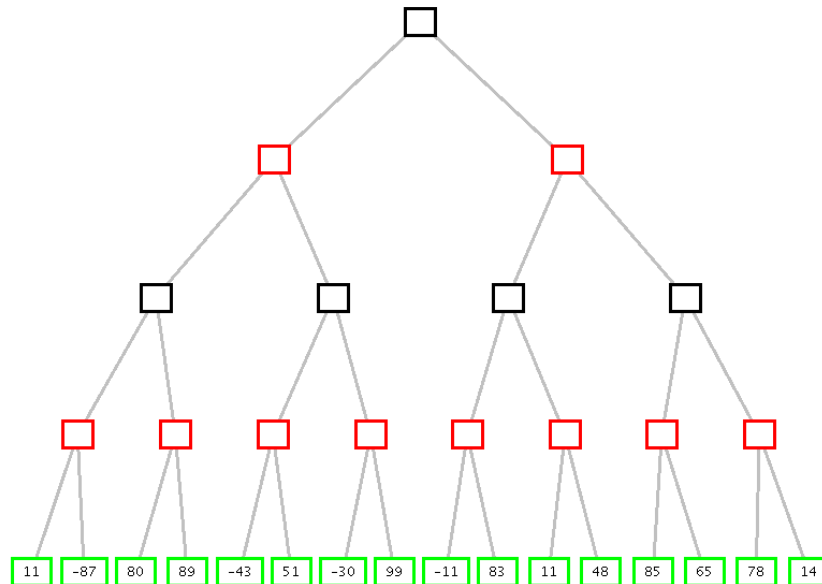
Esta PEC pretende evaluar vuestros conocimientos sobre juegos, su formalización y estrategias relacionadas con la búsqueda de soluciones para juegos.

Descripción de la PEC/práctica a realizar

Esta PEC tiene como objeto los algoritmos de minimax y poda alfa beta, y su implementación en Common Lisp.

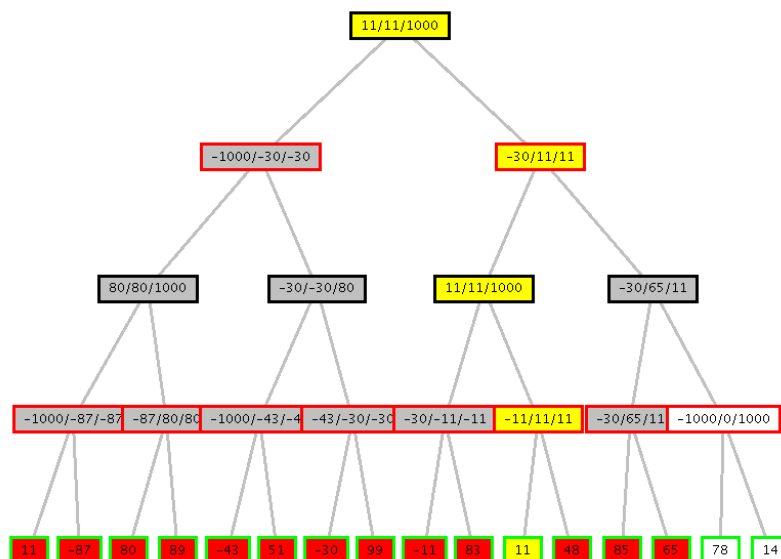
Pregunta 1 (30%):

Suponiendo que el nodo raíz es un nodo max, aplica el algoritmo de la poda alfa-beta sobre este árbol:



Indicad qué rama escogerá el jugador y qué ramas serán podadas.

Solución:



En amarillo la rama escogida. En blanco la rama podada (ignorar los valores en los nodos, pueden ser incorrectos)

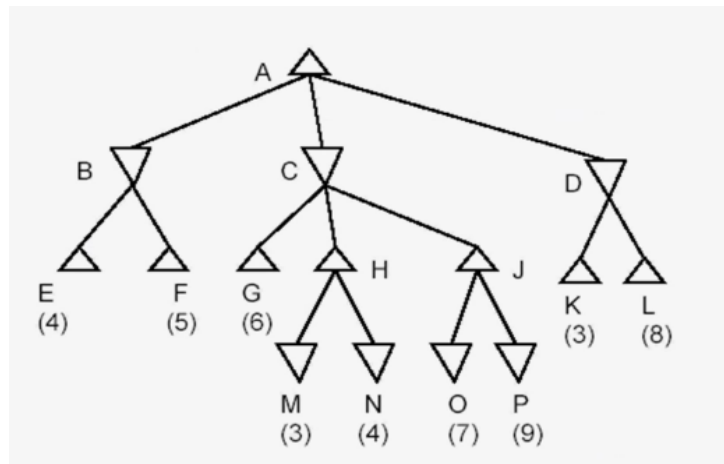
Ahora, intentemos automatizar el proceso.

Representaremos los árboles de juegos en Common Lisp de una manera muy sencilla, considerando tan sólo la puntuación recibida en las hojas del árbol. Así pues, un árbol es o bien *un número*, o bien *una lista* de la siguiente forma:

`(max/min (hijo1 hijo2 . . . hijon))`

es decir, una lista de dos elementos, donde el primer elemento es una etiqueta que puede ser **max** o **min**, y el segundo elemento es una lista de hijos (donde cada hijo es un árbol). Cada nodo puede tener un número arbitrario de hijos. El formato es un poco redundante, ya que una vez sabemos si el nodo raíz es **max** o **min** ya no es necesaria ninguna etiqueta más., pero así los algoritmos son más sencillos.

Por ejemplo, suponiendo que el nodo raíz es un nodo **max**, el siguiente árbol:



sería representado mediante la lista siguiente:

```

(max (min (4 5)
  (min (6
    (max (3 4))
    (max (7 9))))
  (min (3 8)))))

```

(el ejemplo es: <https://www.youtube.com/watch?v=Ewh-rF7KSEg>).

Pregunta 2 (20%): Considerando esta representación de los árboles, definid las siguientes funciones en Common Lisp:

(**hoja arbol**) retorna un booleano, que nos indica si el árbol es una hoja (un número) o no.

(**evalua arbol**) retorna un número si el árbol es una hoja, en otro caso retorna **nil**

(**nodo-min arbol**) retorna un booleano, indica si la raíz del árbol es **min**

(**nodo-max arbol**) retorna un booleano, indica si la raíz del árbol es **max**

(**hijos arbol**) retorna la lista de hijos si el árbol no es una hoja, en otro caso retorna **nil**

Solución:

```
(defun hoja (arbol)
  (numberp arbol))
```

```
(defun evalua (arbol)
  (if (hoja arbol) arbol))
```

```
(defun nodo-min (arbol)
  (eq1 'min (car arbol)))
```

```
(defun nodo-max (arbol)
  (eq1 'max (car arbol)))
```

```
(defun hijos (arbol)
  (if (not (hoja arbol)) (cadr arbol)))
```

Pregunta 3 (20%): Considerando esta representación de los árboles, implementad el algoritmo minimax en Common Lisp, donde el valor que retorna el programa es el valor de la hoja correspondiente al movimiento deseado. Por ejemplo, si llamamos ***tree-levels*** al árbol del ejemplo haciendo:

```
(defparameter *tree-levels*  
  '(max ((min (4 5))  
            (min (6  
                  (max (3 4))  
                  (max (7 9))))  
          (min (3 8)))))
```

el valor retornado por `(minimax *tree-levels*)` debería ser **4** (somos conscientes que este resultado retornado es ambiguo, pero simplifica mucho la realización del programa).

Solución:

```
(defun minimax (arbol)  
  (cond  
    ((hoja arbol) (evalua arbol))  
    ((nodo-min arbol)  
     (apply #'min (mapcar #'minimax (hijos arbol))))  
    ((nodo-max arbol)  
     (apply #'max (mapcar #'minimax (hijos arbol))))))
```

Pregunta 4 (30%): Finalmente, quisiéramos implementar el algoritmo en pseudo-código de la poda alfa-beta de los materiales (página 85 del módulo 2) en Common Lisp. Para simplificar, el programa deberá escribir por pantalla el valor de las hojas visitadas (por lo tanto, las hojas en ramas podadas no aparecerán, de donde podremos deducir qué ramas se han podado) y deberá devolver, igual que el programa de la pregunta 3, el valor de la hoja correspondiente al movimiento deseado. Por ejemplo, haciendo

```
(minimax-alpha-beta *tree-levels* -100 100)
```

debería escribirse por pantalla 4 5 6 3 4 3 y debería retornar 4.

Considerad partir del siguiente fragmento de programa, rellenando aquello que está indicado:

```
(defun minimax-alpha-beta (nodo alpha beta)
  ;; Valores iniciales:
  ;; alpha debería ser -infinito, beta debería ser +infinito
  (cond
    ((hoja nodo)
     (let ((val (evalua nodo)))
       [...]))
    ((nodo-min nodo)
     (let ((beta-tmp beta))
       (do ((ch (hijos nodo) (cdr ch)))
         ((or (null ch) [...]) beta-tmp)
         (let ((r [...]))
           (if [...] (setf beta-tmp r))))))
    ((nodo-max nodo)
     (let ((alpha-tmp alpha))
       (do ((ch (hijos nodo) (cdr ch)))
         ((or (null ch) [...]) alpha-tmp)
         (let ((r [...]))
           (if [...] (setf alpha-tmp r))))))))))
```

Solución:

Fijémonos que es una transcripción *literal* del algoritmo de los materiales:

```
(defun minimax-alpha-beta (nodo alpha beta)
  ;; Valores iniciales:
  ;; alpha debería ser -infinito
  ;; beta debería ser +infinito
  (cond
    ((hoja nodo)
     (let ((val (evalua nodo)))
       (format t "~A " val)
       val))
    ((nodo-min nodo)
     (let ((beta-tmp beta))
       (do ((ch (hijos nodo) (cdr ch)))
         ((or (null ch) (<= beta-tmp alpha)) beta-tmp)
        (let ((r (minimax-alpha-beta (car ch) alpha beta-tmp)))
          (if (< r beta-tmp) (setf beta-tmp r))))))
    ((nodo-max nodo)
     (let ((alpha-tmp alpha))
       (do ((ch (hijos nodo) (cdr ch)))
         ((or (null ch) (<= beta alpha-tmp)) alpha-tmp)
        (let ((r (minimax-alpha-beta (car ch) alpha-tmp beta)))
          (if (< alpha-tmp r) (setf alpha-tmp r))))))))
```

Recursos

Para realizar esta PEC el material imprescindible es el tema 6 del Módulo 2.

Criterios de valoración

Indicados en el enunciado.

Formato y fecha de envío

Para dudas y aclaraciones sobre el enunciado, dirigiros al consultor responsable del aula.

Hay que entregar la solución en un archivo **PDF**. Adjuntar el fichero a un mensaje en el apartado Entrega y Registro de EC (REC).

El nombre del archivo debe ser Apellidos_Nombre_IA_PEC2 con la extensión. pdf (PDF).

La fecha límite de entrega es el: **14 de Abril** (a las 24 horas, más o menos).

Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.

Nota: **Propiedad intelectual**

A menudo es inevitable, al producir una obra multimedia, hacer uso de recursos creados por terceras personas. Es por tanto comprensible hacerlo en el marco de una práctica de los estudios de Informática, siempre que se documente claramente y no suponga plagio en la práctica.

Por lo tanto, al presentar una práctica que haga uso de recursos ajenos, se presentará junto con ella un documento en el que se detallen todos ellos, especificando el nombre de cada recurso, su autor, el lugar donde se obtuvo y el su estatus legal: si la obra está protegida por copyright o se acoge a alguna otra licencia de uso (Creative Commons, licencia GNU, GPL ...). El estudiante deberá asegurarse de que la licencia que sea no impide específicamente su uso en el marco de la práctica. En caso de no encontrar la información correspondiente deberá asumir que la obra está protegida por copyright. Deberán, además, adjuntar los archivos originales cuando las obras utilizadas sean digitales, y su código fuente esté corresponde.